

# *Práctica de Planificación*

Intel·ligència Artificial

2025/2026 1Q

Facultat d'Informàtica de Barcelona

Iu Franquesa Tarté

Junzhe Xia

Yaoyao Guo Zhang

# Índice

|                                                     |           |
|-----------------------------------------------------|-----------|
| <b>1. Introducción</b>                              | <b>3</b>  |
| <b>2. Modelado de dominios</b>                      | <b>4</b>  |
| Dominio Básico                                      | 4         |
| Extensión 1                                         | 4         |
| Extensión 3                                         | 4         |
| Extensión 4                                         | 4         |
| <b>3. Modelado de problemas</b>                     | <b>6</b>  |
| <b>4. Desarrollo de modelos</b>                     | <b>7</b>  |
| <b>5. Juegos de prueba</b>                          | <b>8</b>  |
| Básico                                              | 9         |
| Prueba 1                                            | 9         |
| Prueba 2                                            | 9         |
| Extensió 1                                          | 10        |
| Prueba 1                                            | 10        |
| Prueba 2                                            | 10        |
| Extensió 3                                          | 10        |
| Prueba 1                                            | 11        |
| Prueba 2                                            | 11        |
| Extensió 4                                          | 12        |
| Prueba 1                                            | 12        |
| Prueba 2                                            | 12        |
| Conclusiones                                        | 13        |
| <b>6. Estudio de complejidad</b>                    | <b>14</b> |
| Básico                                              | 14        |
| Extensión 1                                         | 14        |
| Extensión 3                                         | 15        |
| Extensión 4                                         | 15        |
| <b>ANEXOS</b>                                       | <b>16</b> |
| [Anexo-1] Documento de estudio de complejidad       | 16        |
| [Anexo-2] Hoja de cálculo de estudio de complejidad | 16        |

# 1. Introducción

Esta práctica aborda un problema de **asignación de recursos con restricciones temporales**, concretamente la gestión de reservas de un hotel, que resulta especialmente adecuado para ser modelado mediante planificación.

Cada reserva puede verse como una tarea que debe ser decidida (asignada o descartada) respetando restricciones de capacidad y solapamiento, y el objetivo global consiste en encontrar una secuencia de acciones que conduzca desde un estado inicial a un estado objetivo válido u óptimo. La planificación permite expresar de forma declarativa tanto las acciones disponibles como las condiciones bajo las cuales pueden aplicarse, delegando en el planificador la búsqueda de la mejor solución.

A lo largo de la práctica se trabajan conceptos fundamentales de la planificación, como la definición de dominios y problemas en PDDL, el uso de predicados y flujos numéricos, las precondiciones y efectos (incluyendo cuantificadores), y la optimización mediante funciones de coste.

Asimismo, se introduce el uso de métricas para manejar objetivos múltiples y priorizados, lo que permite modelar situaciones realistas en las que no solo importa alcanzar un objetivo, sino hacerlo de la mejor manera posible según criterios definidos.

## 2. Modelado de dominios

### Dominio Básico

Para poder representar el problema, se ha requerido de tipos de objetos y fluentes. Los tipos utilizados son los siguientes:

- habitacio, para representar una habitación del hotel
- reserva, para representar una reserva de un cliente
- dia, para representar los días en los que se quiere reservar

Como predicados:

- Un predicado que nos indica si una reserva ha sido servida ya o no
- Un predicado que nos indica a qué habitación se ha asignado una reserva
- Un predicado que nos indica si una habitación está ocupada o no en un día determinado
- Un predicado que nos indica si una reserva se aplica en un día determinado

Como funciones:

- Una función que nos indica la capacidad de una habitación (número de personas que caben)
- Una función que nos indica el número de personas de una reserva

Como acciones:

- Una acción que asigna una habitación a una reserva. Esta acción comprueba que la reserva no está servida, que las capacidades se cumplen, y que todos los días que se quieren reservar, la habitación se encuentra libre en esos días. Una vez hecho la comprobación, marca la reserva como servida, asigna la habitación a la reserva y marca como ocupada todos los días de la reserva a la habitación

### Extensión 1

Con respecto al dominio básico, añadimos lo siguiente:

- Un nuevo predicado que nos indica que una reserva ha sido rechazada
- Una nueva función que nos indica el coste total de la asignación de las reservas
- En la acción de asignar una habitación a una reserva se comprueba además que la reserva no haya sido rechazada
- Una nueva acción que rechaza una reserva. Esta acción comprueba que no haya sido servida y que no haya sido rechazada ya previamente. Una vez hecho la comprobación, también marca la reserva como rechazada y aumenta el coste total de la asignación de las reservas en 1000 unidades

### Extensión 3

Con respecto al dominio de la extensión 1, añadimos lo siguiente:

- En la acción de asignar una habitación a una reserva, una vez comprobado las restricciones, se suma al coste total la diferencia entre la capacidad de la habitación y el número de personas de la reserva.

### Extensión 4

Con respecto al dominio de la extensión 3, añadimos lo siguiente:

- Un nuevo predicado que nos indica si una habitación ya ha sido utilizada previamente o no
- En la función de asignar una reserva a una habitación, una vez comprobado las restricciones, también suma 1000 al coste total en caso de que la habitación asignada no se haya usado previamente (coste de abrir una nueva habitación). El hecho de rechazar una reserva ahora tiene una penalización de 1000000

### 3. Modelado de problemas

El modelado de los problemas se ha realizado de forma coherente y homogénea para todas las extensiones, definiendo explícitamente los **objetos**, el **estado inicial** y el **estado objetivo** en cada fichero de problema PDDL.

Los objetos representan las entidades relevantes del dominio: habitaciones, reservas y días, lo que permite discretizar el tiempo y expresar de forma clara las restricciones temporales.

En el estado inicial se especifican las capacidades de las habitaciones, el número de personas de cada reserva y los días concretos en los que cada reserva está activa, mediante predicados que indican la estancia en cada día. El estado inicial también incluye los valores iniciales de los flujos numéricos necesarios en cada extensión, como el número de reservas no asignadas o el coste acumulado, según corresponda.

Por su parte, el estado objetivo varía según el nivel: en el modelo básico se exige que todas las reservas estén asignadas, mientras que en las extensiones se relaja este requisito y se fuerza únicamente a que todas las reservas hayan sido procesadas, permitiendo así la optimización mediante métricas. Esto garantiza que el planificador tome una decisión para cada reserva, ya sea asignarla o descartarla.

Finalmente, en las extensiones más avanzadas, el modelado del problema incorpora funciones métricas en la definición del objetivo, lo que permite optimizar distintos criterios como el número de reservas asignadas o el desperdicio de plazas. De este modo, el problema no solo consiste en alcanzar un estado final válido, sino en hacerlo de la mejor forma posible según los objetivos definidos, manteniendo siempre una separación clara entre la definición del dominio y la especificación concreta de cada instancia del problema.

## 4. Desarrollo de modelos

El desarrollo de los modelos se realizó de forma iterativa, partiendo de un modelo base correcto y resoluble, y ampliándolo progresivamente según los requisitos de cada nivel.

En primer lugar se construyó el **nivel básico**, centrado únicamente en la asignación válida de reservas a habitaciones, garantizando las restricciones de capacidad y no solapamiento temporal. Este modelo sirvió como referencia estable para comprobar la corrección semántica del dominio y facilitar la detección temprana de errores de modelado. A partir de este núcleo, se introdujeron las extensiones de manera incremental.

En la **Extensión 1** se añadieron flujos numéricos y acciones adicionales para permitir reservas no asignadas y optimizar el número total de asignaciones.

En la **Extensión 3** se incorporó un segundo criterio de optimización (desperdicio de plazas), ajustando la función métrica para priorizar correctamente los objetivos. Este enfoque incremental permitió reutilizar la mayor parte del modelado previo y validar cada ampliación de forma controlada antes de pasar a la siguiente.

En la **Extensión 4**, el modelo se amplió manteniendo la misma filosofía incremental, añadiendo nuevas restricciones y/o criterios sin modificar la estructura básica del dominio. Se reutilizaron las acciones y predicados ya existentes, incorporando únicamente los elementos necesarios para representar el nuevo comportamiento solicitado.

## 5. Juegos de prueba

Para los juegos de **prueba 1** de cada extensión, hemos creado un conjunto de problemas de prueba incrementales basados en el mismo escenario. El problema plantea un hotel con 4 habitaciones de diferentes capacidades (2, 3, 4 y 2 personas) que debe gestionar 5 reservas. La complejidad radica en que varias reservas se solapan temporalmente:

- r1: 2 personas, días 1-2
- r2: 3 personas, días 2-3
- r3: 2 personas, días 1-3 (solapa con r1 y r2)
- r4: 4 personas, días 3-5
- r5: 2 personas, días 1-2 (solapa con r1 y r3)

Este diseño hace que no sea trivial encontrar una asignación óptima y permite probar las diferentes estrategias de optimización de cada extensión.

Los ficheros de prueba usados son:

```
None
problem-basic-test.pddl
problem-ext1-test.pddl
problem-ext3-test.pddl
problem-ext4-test.pddl
```

Comando para ejecutar:

```
Shell
./Metric-FF/ff -O -o ./domain-basic.pddl -f ./jocs-prova/problem-basic-test.pddl
./Metric-FF/ff -O -o ./domain-ext1.pddl -f ./jocs-prova/problem-ext1-test.pddl
```

Para los juegos de **prueba 2**, se han usado los generados aleatoriamente por el generador automático. Comandos usados:

```
Shell
python3 ./generate_problems.py 10 --level basic
./Metric-FF/ff -O -o ./domain-basic.pddl -f ./problem-basic-auto-10.pddl
```

Ficheros usados:

```
None
Joc_Prova2_Basic.pddl
Joc_Prova2_Ext1.pddl
Joc_Prova2_Ext3.pddl
Joc_Prova2_Ext4.pddl
```



# Básico

## Objetivo:

Probar que el planificador puede asignar todas las reservas respetando las restricciones básicas de capacidad y disponibilidad temporal de las habitaciones.

## Qué probamos:

- Que cada habitación solo se asigna a una reserva por cada día
- Que la capacidad de la habitación es suficiente para las personas de la reserva
- Que el planificador encuentra un plan válido para servir todas las reservas

## Prueba 1

### Resultado esperado:

El planificador debería asignar:

- h1 o h4 (capacidad 2) para r1, r3 o r5
- h2 (capacidad 3) para r2
- h3 (capacidad 4) para r4

Debido a los solapamientos (r1, r3 y r5 coinciden en días 1-2), el planificador necesita usar al menos 3 habitaciones diferentes para estas tres reservas. El plan debe tener 5 acciones de tipo "asignar", una para cada reserva.

```
./Metric-FF/ff -O -o domain-basic.pddl -f problem-basic-test.pddl
```

```
ff: found legal plan as follows

step    0: ASSIGNAR R3 H4
        1: ASSIGNAR R4 H3
        2: ASSIGNAR R1 H3
        3: ASSIGNAR R2 H2
        4: ASSIGNAR R5 H1
```

## Prueba 2

### Resultado esperado:

Es posible encontrar una asignación para todas las reservas de tal forma que no se solapen.

```
step    0: ASSIGNAR R9 H5
        1: ASSIGNAR R3 H5
        2: ASSIGNAR R6 H5
        3: ASSIGNAR R1 H5
        4: ASSIGNAR R5 H4
        5: ASSIGNAR R2 H3
        6: ASSIGNAR R10 H3
        7: ASSIGNAR R8 H3
        8: ASSIGNAR R4 H2
        9: ASSIGNAR R7 H1
```

## Extensió 1

### Objetivo:

Probar la capacidad de rechazar reservas cuando es necesario y verificar que el planificador minimiza el número de rechazos.

### Qué probamos:

- La acción de rebutjar funciona correctamente
- El planificador prioriza maximizar el número de reservas servidas (minimizar rechazos)
- El coste de rechazar una reserva (1000) es lo suficientemente alto como para que se priorice servir las

### Efecto de la ponderación:

El coste de 1000 por rechazo hace que el planificador intente servir el máximo número de reservas posible. En este problema, como tenemos 4 habitaciones y 5 reservas con solapamientos, el planificador debería intentar servir todas las reservas si es posible.

## Prueba 1

### Resultado esperado:

La reserva 4 es imposible de asignar ya que tiene una capacidad que supera a todas las capacidades de las diversas habitaciones. Las demás habitaciones pueden ser asignadas.

```
step    0: ASSIGNAR R5 H1
        1: ASSIGNAR R3 H2
        2: ASSIGNAR R2 H2
        3: ASSIGNAR R1 H3
        4: REBUTJAR R4
        5: REACH-GOAL
```

## Prueba 2

### Resultado esperado:

Es posible encontrar una asignación para todas las reservas de tal forma que no se solapen.

```
step    0: ASSIGNAR R10 H1
        1: ASSIGNAR R9 H1
        2: ASSIGNAR R8 H1
        3: ASSIGNAR R7 H3
        4: ASSIGNAR R6 H1
        5: ASSIGNAR R5 H2
        6: ASSIGNAR R4 H2
        7: ASSIGNAR R3 H1
        8: ASSIGNAR R2 H1
        9: ASSIGNAR R1 H4
       10: REACH-GOAL
```

## Extensió 3

### Objetivo:

Probar la minimización del desperdicio de capacidad, además de minimizar rechazos.

### Qué probamos:

- Que el planificador asigna habitaciones ajustando mejor la capacidad a las necesidades
- El equilibrio entre no rechazar reservas y minimizar desperdicio
- Cómo afecta el desperdicio a la elección de habitaciones cuando hay varias opciones válidas

### Efecto de la ponderación:

Ahora tenemos dos criterios:

- Rechazar una reserva: +1000 al coste
- Desperdicio de capacidad: +(capacidad - personas) por cada asignación

Con estos pesos, rechazar sigue siendo mucho más caro que el desperdicio.

## Prueba 1

### Resultado esperado:

El planificador debería:

1. Primero asegurar servir el máximo número de reservas
2. Entre opciones válidas, elegir habitaciones que minimicen el desperdicio

Las reservas r4 y r5 pueden asignarse a cualquier habitación, pero no pueden estar en la misma habitación (se solapan). Lo ideal es que r4 se asignara a h1 y r5 a h2 o al revés para poder optimizar el resultado.

```
step    0: ASSIGNAR R5 H1
        1: ASSIGNAR R4 H2
        2: ASSIGNAR R3 H4
        3: ASSIGNAR R2 H3
        4: ASSIGNAR R1 H1
        5: REACH-GOAL
```

## Prueba 2

### Resultado esperado:

Es posible encontrar una asignación para todas las reservas de tal forma que no se solapen, minimizando el número de reservas no asignadas y la diferencia entre la capacidad de las habitaciones y las personas de la reserva.

```
step    0: ASSIGNAR R10 H2
        1: ASSIGNAR R9 H4
        2: ASSIGNAR R8 H1
        3: ASSIGNAR R7 H3
        4: ASSIGNAR R6 H4
        5: ASSIGNAR R5 H4
        6: ASSIGNAR R3 H4
        7: ASSIGNAR R1 H5
        8: ASSIGNAR R2 H1
        9: ASSIGNAR R4 H1
       10: REACH-GOAL
```

## Extensió 4

### Objetivo:

Probar la minimización del número de habitaciones utilizadas, manteniendo las prioridades anteriores.

### Qué probamos:

- Que el planificador reutiliza habitaciones cuando es posible (para reservas en días diferentes)
- La jerarquía de prioridades: no rechazar > menos habitaciones > menos desperdicio
- La funcionalidad del predicado ja-ocupada

### Efecto de la ponderación:

Ahora tenemos tres criterios jerárquicos:

- Rechazar una reserva: +1000000 al coste (prioridad máxima)
- Abrir una habitación nueva: +1000 la primera vez que se usa
- Desperdicio de capacidad: +(capacidad - personas) por asignación

La jerarquía garantiza que primero se intenta no rechazar, luego se minimiza el número de habitaciones, y finalmente se minimiza el desperdicio.

## Prueba 1

### Resultado esperado:

El planificador debería:

1. No rechazar ninguna reserva (si es posible)
2. Intentar reutilizar habitaciones para reservas que no se solapan temporalmente
3. Minimizar el desperdicio en las asignaciones

En este caso solo se pueden usar las habitaciones h1 y h2. Como no hay solapamiento debería de elegir siempre h2, ya que todas las reservas se pueden asignar a esa habitación y no es necesario usar la habitación h1 también.

```
step    0: ASSIGNAR R5 H2
        1: ASSIGNAR R3 H2
        2: ASSIGNAR R4 H2
        3: ASSIGNAR R2 H2
        4: ASSIGNAR R1 H2
        5: REACH-GOAL
```

## Prueba 2

### Resultado esperado:

Es posible encontrar una asignación para todas las reservas de tal forma que no se solapen, minimizando el número de reservas no asignadas, la diferencia entre la capacidad de las habitaciones y las personas de la reserva y las habitaciones usadas.

```
step    0: ASSIGNAR R10 H2
        1: ASSIGNAR R9 H2
        2: ASSIGNAR R6 H2
        3: ASSIGNAR R5 H2
        4: ASSIGNAR R3 H2
        5: ASSIGNAR R8 H1
        6: ASSIGNAR R2 H1
        7: ASSIGNAR R4 H1
        8: ASSIGNAR R7 H3
        9: ASSIGNAR R1 H4
       10: REACH-GOAL
```

## Conclusiones

Los problemas de prueba diseñados permiten observar cómo cada extensión añade un nuevo criterio de optimización:

1. Básico: solo busca una solución válida
2. Ext1: prioriza maximizar reservas servidas
3. Ext3: añade eficiencia en el uso de capacidad
4. Ext4: añade eficiencia en el número de habitaciones usadas

La ponderación de costes establece una jerarquía clara que refleja las prioridades típicas de un hotel: primero intentar servir a todas las peticiones de reserva, luego reducir costes operativos (menos habitaciones abiertas), y finalmente optimizar recursos (capacidad).

## 6. Estudio de complejidad

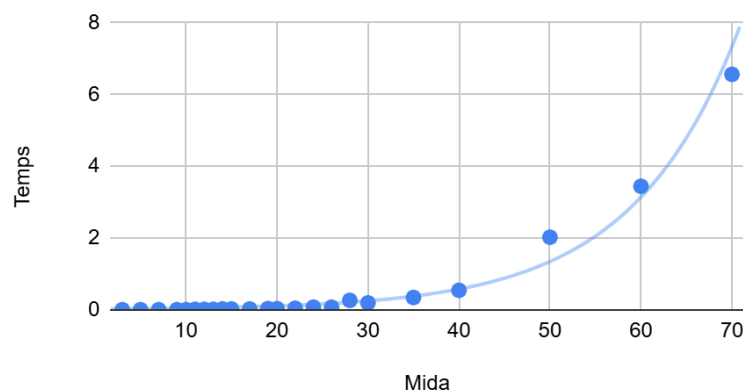
Se ha hecho un estudio de la complejidad temporal con la ayuda del generador aleatorio de juegos de prueba, que nos ha permitido probar con casos de tamaño creciente.

Los comandos y ficheros usados, así como los resultados, se encuentran en los anexos 1 y 2.

### Básico

Observamos que el tiempo de ejecución se ve de forma polinómica. Esto es porque no hay optimizaciones, por tanto no hay que explorar nada. Sin embargo, a medida que vamos aumentando el tamaño de entrada, las comprobaciones de la función asignar va aumentando en consecuencia, ya que este es lineal al número de días que hay en la base de datos.

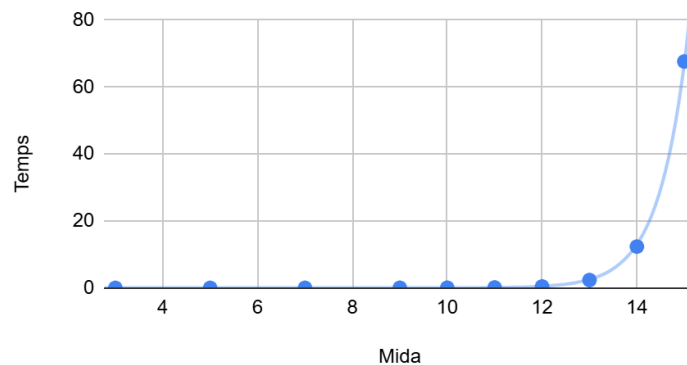
Temps i Mida - Bàsic



### Extensión 1

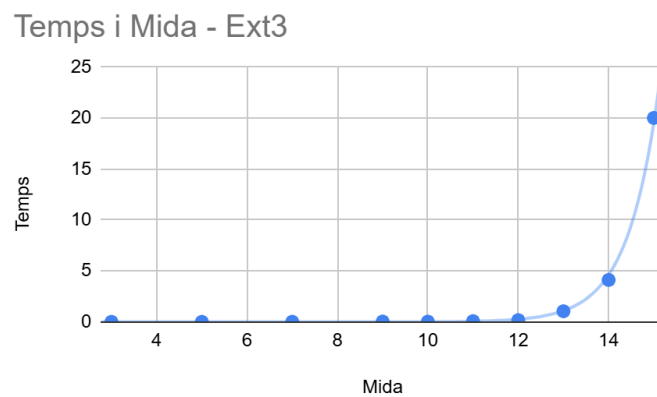
El tiempo de ejecución es exponencial una vez superado las 14 reservas. El número de comprobaciones para la función de asignar es lineal al número de días que hay en la base de datos. Al igual, al tener que optimizar y mirar las diferentes ramas que existen, es de esperarse que el tiempo aumente de forma exponencial.

Temps i Mida - Ext1



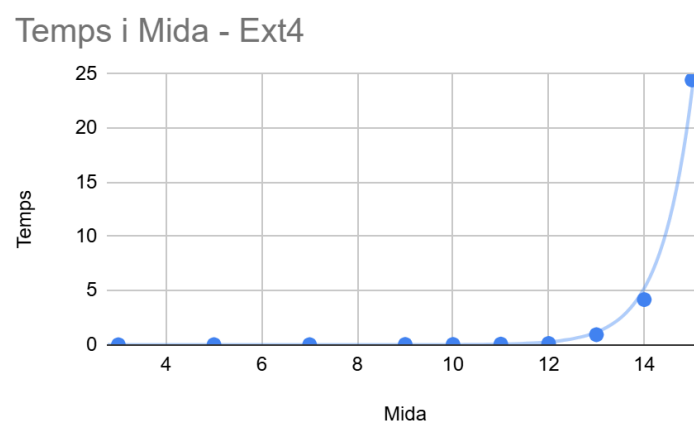
## Extensión 3

Observamos que el tiempo de ejecución crece de forma exponencial una vez superado las 14 reservas. Esto se debe a la cantidad de ramas que hay que explorar para poder encontrar las soluciones óptimas. Además el número de comprobaciones que hay que hacer antes de ejecutar las acciones también crece. Por ejemplo, la acción de asignar tiene que realizar una comprobación para todos los días que existen en la base de datos.



## Extensión 4

Observamos que de forma parecida, a partir de las 13-14 reservas, el tiempo de ejecución crece de forma exponencial. La razón es muy parecida a la de la extensión 3. No obstante en este caso, hay más acciones que pueden realizar un cambio sobre el parámetro a optimizar, por tanto, más ramas a explorar.



# ANEXOS

[Anexo-1] Documento de estudio de complejidad

<https://docs.google.com/document/d/19phxiAiI-k4IfTDBT5x9ajcrSjcVBbQ1q-3RdhOIJI/edit?usp=sharing>

[Anexo-2] Hoja de cálculo de estudio de complejidad

<https://docs.google.com/spreadsheets/d/1ch8kKD8BATUsy1ulJ4VcxW2G-r5PGOOpsBz2f9TYvDY/edit?usp=sharing>